

JAVA

```
class Arithmetic{ //
    Arithmetic(){
        System.out.println("COnstructor called");
    }
    Arithmetic(int a , int b){
        System.out.println(a+b);
    }
    public void add(int a, int b){
        System.out.println(a+b);
    }
    public void add(int a, int b, int c){
        System.out.println(a+b+c);
    }
}

class Payment{

    void pay(){
        System.out.println("Payment paid");
    }
}

class UPI extends Payment{
    void pay(){
        System.out.println("Payment paid using UPI");
    }
}

class Parent{

    int salary = 50000;
}

class Child extends Parent{

}

class Child1 extends Parent{

}

class Bank{
```

```

    private int accountNumber = 123;

    public void getter(){
        System.out.println(accountNumber);
    }

    public void setter(int acct){
        accountNumber = acct;
    }
}

public class Day4 {

    public static void main(String[] args) {

        Bank bank = new Bank();

        // System.out.println(bank.accountNumber);
        bank.setter(300);
        bank.getter();

        Arithmetic arith = new Arithmetic(10,20);

        arith.add(10, 20);
        arith.add(10,20,30);

        Child obj = new Child();

        System.out.println(obj.salary);

        // System.out.println("String");

        // string is immutable

```

```

        // stringBuilder and stringBuffer => mutable

        // StringBuilder sb = new StringBuilder("Hello world");

        // sb.append(" Java");
        // sb.insert(0, "Java ");
        // sb.delete(0, 5); // n-1 (5-1) => 4
        // System.out.println(sb);

        // StringBuffer sb1 = new StringBuffer("FIIT");

        // // synchronized , thread-safe

        // sb1.append(" Java");
        // sb1.insert(0, "Institute ");

        // System.out.println(sb1);

        // listen => 'l', 'i', 's', 't'

    }
}

```

```

abstract class Abstraction{

    abstract void display(); // incomplete method
    abstract void fiit();

    void showDetails(){
        System.out.println("Show details");
    }
}

class Child extends Abstraction{

    void display(){
        System.out.println("Display method from child class");
    }
    void fiit(){
        System.out.println("Fiit ");
    }
}
// this and super

```

```

class Student{

    String name;

    Student(String name){
        this.name = name;
        System.out.println("Student COnstructor called");
        System.out.println(this.name);
    }
}

class PT extends Student{

    PT(){
        super("Harish"); // parent constructor
        System.out.println("PT constructor called");
    }
}

public class Day4a {

    public static void main(String[] args) {

        PT obj = new PT();

        System.out.println("Abstraction");
    }
}

```

Theory

- **Constructors**
 - Special methods used to initialize objects.
 - Can be **default** or **parameterized**.
 - Called automatically when an object is created.
- **Method Overloading**
 - Same method name, different parameter lists.
 - Achieved at **compile-time**.

- **Method Overriding**
 - Child class provides a new implementation of a parent class method.
 - Achieved at **runtime** (polymorphism).
- **Inheritance**
 - Mechanism to acquire properties and methods of another class.
 - `extends` keyword is used.
- **Encapsulation**
 - Wrapping data (fields) and methods together.
 - Achieved using **private variables** and **public getters/setters**.
- **Abstraction**
 - Hiding implementation details and showing only functionality.
 - Achieved using **abstract classes** and **interfaces**.
- **this and super**
 - `this` → refers to current object.
 - `super` → refers to parent class object or constructor.
- **StringBuilder vs StringBuffer**
 - Both are mutable.
 - `StringBuilder` → not synchronized, faster.
 - `StringBuffer` → synchronized, thread-safe.



Coding Round Questions

- Constructor overloading → Write a class with both default and parameterized constructors.
- Method overloading → Implement an `add()` method that works with two and three parameters.
- Method overriding → Create a parent `Payment` class and override its `pay()` method in a child `UPI` class.
- Inheritance → Demonstrate single inheritance where Child inherits salary from Parent.
- Encapsulation → Create a `Bank` class with a private `accountNumber` and public `getter/setter` methods.

- Abstraction → Define an abstract class with abstract methods and implement them in a child class.
- this and super → Demonstrate how this refers to current object and super calls parent constructor.
- StringBuilder vs StringBuffer → Show how both can append and insert strings, and explain thread-safety differences.

⚡ Machine Coding Round Challenge

Problem: Banking System with OOP Concepts

Requirements:

1. Create a BankAccount class with:
 - Private fields: accountNumber, balance.
 - Public methods: deposit(), withdraw(), getBalance().
2. Demonstrate **constructor overloading**:
 - Default constructor initializes balance = 0.
 - Parameterized constructor initializes with given balance.
3. Demonstrate **method overloading**:
 - deposit(int amount) → deposit money.
 - deposit(int amount, double interest) → deposit with interest.
4. Demonstrate **method overriding**:
 - Create a SavingsAccount class that overrides withdraw() to restrict withdrawals if balance < 1000.
5. Demonstrate **abstraction**:
 - Create an abstract class Loan with abstract method calculateEMI().
 - Implement it in HomeLoan and CarLoan classes.
6. Demonstrate **encapsulation**:
 - Keep accountNumber private and use getter/setter.